

```
In [1]: #Import Libraries
```

```
In [2]: import hashlib
import time
```

```
In [3]: def sha256_string(s: str) -> str:

    #Compute the SHA-256 hash of a string.

    return hashlib.sha256(s.encode("utf-8")).hexdigest()

def sha256_file(path: str) -> str:

    #Compute the SHA-256 hash of a file.

    h = hashlib.sha256()

    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(8192), b''):
            h.update(chunk)

    return h.hexdigest()
```

```
In [4]: # Compute the SHA-256 digest of a string
text = "Hello Blockchain"
print("String:", text)
print("SHA-256:", sha256_string(text))

print()
```

String: Hello Blockchain
SHA-256: 7cf88f2ee398c0b7c0e760a1dccaf3571e0baccf310f11fe3bdfd0b09675ea75

```
In [5]: print("File:", "Assignment 1.docx")

print(
    "SHA-256:",
    sha256_file(
        r"C:\Users\USER\OneDrive\Documents\Mcom in Financial Engineering\Semester 1
    )
)
```

File: Assignment 1.docx
SHA-256: fa5da5bc07767401931e48feba14906ec2d51502bacb166d1ab0c146d17d018f

```
In [6]: # Part (b): Avalanche Effect

original = "Transaction1"
modified = "transaction1"

print("Original String :", original)
print("Original Hash   :", sha256_string(original))
```

```
print()

print("Modified String :", modified)
print("Modified Hash   :", sha256_string(modified))
```

Original String : Transaction1

Original Hash : 08e80a506bc7bb040dde684681ebd70dbc8d93fab268e090956e5389dd47fdca

Modified String : transaction1

Modified Hash : bde4693e55a336ff81ab238ce20cae1dd9c8ba03b9b8f43963f5569bf3cf5229

In [7]: *# Part (c): Proof of Work*

```
def proof_of_work(message: str, n: int):

    prefix = "0" * n
    nonce = 0

    start = time.perf_counter()

    while True:

        digest = sha256_string(f"{message}{nonce}")

        if digest.startswith(prefix):

            end = time.perf_counter()

            return {
                "nonce": nonce,
                "digest": digest,
                "attempts": nonce + 1,
                "seconds": end - start
            }

        nonce += 1
```

In [8]: *# For n=3*

```
print("Proof-of-Work (n = 3)")
result = proof_of_work("Blockchain", 3)

for key, value in result.items():
    print(f"{key}: {value}")
```

Proof-of-Work (n = 3)

nonce: 3762

digest: 000cdabccf4e8ce5416c99adc5d70a374fc0c7cdefab9d5d3282fc9be8d15acd

attempts: 3763

seconds: 0.033600400000068475

In [9]: *# For n=4*

```
print("\nProof-of-Work (n = 4)")
result = proof_of_work("Blockchain", 4)
```

```
for key, value in result.items():  
    print(f"{key}: {value}")
```

Proof-of-Work (n = 4)

nonce: 57565

digest: 0000bd847be5ffeaa0f47f2c6b66c4fe28fa71875c2b18d5a7f699028fc94fdc

attempts: 57566

seconds: 0.44197650000023714

In []: